Implementation: Wire the service-booking feature

About this document

This technical companion explains how to implement the feature in the existing Week 4, Lesson 8 PetCareAppMVC project using the supplied database and EJS shells.

service-booking.md is the source of truth for behaviour, messages, scope, and acceptance criteria. This document supplies file responsibilities and integration contracts, not additional user-facing features. If the behavioural spec, schema, or shells conflict, clarify the conflict before changing them.

1. Provided resources and prerequisites

Resource	Use
service-booking.md	Behaviour and acceptance criteria AC1-AC8
db.sql	Table names, columns, relationships, and sample data
db-lesson08-extension.sql	One-time additive setup for an existing Lesson 06 database
services.ejs	Service selection and booking form shell
bookings.ejs	Booking History shell
app.js	Existing Express middleware and route registration
db.js	Shared MySQL connection pool
auth.js	Existing `isLoggedIn` middleware
petModel.js	Existing owned-pet query
navbar.ejs	Existing shared navigation

Start from an application with working login, `req.session.user`, flash messages, EJS rendering, and MySQL connectivity. The provided schema must be available in the intended development database.

Do not automatically execute the SQL setup script. It contains `DROP TABLE` statements. Obtain explicit approval before importing it or resetting any data. Use a separate disposable database for destructive setup and failure tests. Do not copy database credentials into this specification or newly created files.

For students continuing from Lesson 06, provide the extension script instead of reimporting the full reset script. With explicit approval, run it once on their own development database, where the three new tables do not yet exist. It preserves users, pets, and journals and adds the Lesson 08 catalogue. Inspect any table-exists or partial-setup error before continuing; do not blindly rerun it.

2. Files to create or update

Create missing files. If a file already exists, inspect and reuse or complete its implementation rather than overwriting working code.

File	Responsibility
serviceRoutes.js	Declare the three protected routes
serviceController.js	Validate inputs, call models, render pages, and manage messages/redirects
serviceModel.js	Retrieve active database services
serviceBookingModel.js	Transactional booking/payment creation and owned-booking history
app.js	Register the service router once
services.ejs	Bind supplied data to the existing form shell
bookings.ejs	Bind supplied data to the existing history shell
navbar.ejs	Ensure signed-in users can reach services and history

Reuse the existing pet model, pool, authentication middleware, and session/flash configuration. No additional payment controller or payment-provider integration is required for this feature.

3. Routing and middleware contract

Method	Path	Middleware	Controller export
`GET`	`/services`	`isLoggedIn`	`listServices`
`POST`	`/services/book`	`isLoggedIn`	`createBooking`
`GET`	`/bookings`	`isLoggedIn`	`listBookings`

Export an Express router from the service routes module. Mount it at the application root in app.js:

```
app.use(require('./routes/serviceRoutes'));
```

Keep the existing EJS setup, URL-encoded body parser, session middleware, and flash middleware before router registration. Do not mount the router a second time or add a prefix that changes the specified URLs.

Read the owner ID from `req.session.user.id`, never from a body/query field. Unauthenticated requests use the existing middleware's redirect to `/login`.

4. Controller function contracts

Use the existing CommonJS and callback-based MVC conventions.

Export	Inputs	Responsibilities
`listServices(req, res)`	Session user; optional `req.query.petId`	Load owned pets and active services; preselect an owned pet; render `services`

Export	Inputs	Responsibilities
`createBooking(req, res)`	Session user; `petId`, `serviceId`, `bookingDate` from `req.body`	Validate form values; call transactional model; handle success, unavailability, and database errors
`listBookings(req, res)`	Session user	Load owned-booking history; render `bookings`

The controller must:

- Apply the behavioural specification's ID and date checks before saving.
- Build `today` as a local calendar date in `YYYY-MM-DD`. Do not hard-code the classroom example date or use a UTC date that changes the local-day boundary.
- Check that the submitted date exists, not merely that its format matches.
- Treat `petId` query preselection as optional; only select it from the returned owned-pet list. Otherwise use an empty selection.
- Leave ownership/service verification and database price lookup in the model's save operation, so browser choices cannot bypass them.
- Use `req.flash('success', ...)` and `req.flash('error', ...)` with the exact messages in section 7 of the behavioural spec.
- Redirect to `/bookings` only after the model reports a committed booking.
- Return after sending or redirecting; do not send multiple responses.
- Log database failures using the existing `console.error` convention.

On a database save error, the existing implementation redirects to `/services?petId=<submittedPetId>`. Preselection must still verify ownership. Validation and unavailable-item outcomes redirect to `/services`.

For read errors, return HTTP `500` and an explicit message:

Failed operation	Response text
Load pets	`Unable to load pets.`
Load services	`Unable to load services.`
Load history	`Unable to load booking history.`

These read-error texts follow the existing controller and are consistent with the behavioural spec's database-unavailability outcome.

5. Model function contracts

Existing pet query

Reuse `petModel.getPetsByOwner(ownerId, callback)`.

- On success, `callback(null, pets)` returns an array of owned pets.
- Fields consumed by the booking shell are `id`, `name`, and `species`.
- An empty result is `[]`; a database error is passed to the callback.

Active service query

Export `serviceModel.getActiveServices(callback)`.

- Read `id`, `name`, and `price` from the database `services` table.
- Filter by `is_active = 1` and order by `name`.
- Return `callback(null, services)` with an array, or pass the database error.
- Do not substitute the older classroom mock catalogue for database services.

Transactional booking creation

Export:

```
createPendingBooking(ownerId, petId, serviceId, bookingDate, callback)
```

The controller supplies validated numeric IDs and a validated date string. The model verifies ownership and active service availability as part of saving.

Outcome	Callback contract
Both records committed	`callback(null, { bookingId, paymentId })`
Pet/service unavailable or not owned	`callback(null, null)`
Database operation failed	`callback(error)`

Do not disguise database errors as an empty/unavailable result. Invoke the callback exactly once, and only report success after the commit completes.

Booking-history query

Export `serviceBookingModel.getBookingsByOwner(ownerId, callback)`.

- Join `service_bookings` to `pets` and `services`.
- Filter by `pets.owner_id = ownerId`.
- Left join `payments` so a booking without a payment row remains visible.
- Sort by booking creation time descending, then booking ID descending.
- Return `callback(null, bookings)` with an array, or pass the database error.
- Provide the view fields specified in section 7 below.

Use parameterised SQL for all request/session-derived values.

6. Transaction and database integration

Use one connection acquired from the shared pool for the entire transaction. Do not run individual transaction queries through separate pool connections.

1. Acquire a connection and begin a transaction.
2. Create the booking only when the selected pet is owned by `ownerId` and the selected service is active. Use the database service price as `booked_price`.

3. Confirm that exactly one booking was inserted. If no eligible pet/service matches, roll back and return the unavailable outcome.
4. Create exactly one payment linked to the new booking ID. Its amount comes from the booking's stored price, not a request field or a second catalogue price lookup. Set both initial statuses to lowercase `pending`.
5. Commit only after both inserts succeed.
6. On an insert/commit failure, attempt rollback and propagate the failure. Cleanup failures must not be silently presented as success.
7. Release any acquired connection on every terminal path, including a failed transaction start, rollback, or successful commit.

Use the supplied table/column names and foreign keys. Leave `payment_reference` unset (`NULL`) during pending-record creation. Do not add schema changes, duplicate-booking restrictions, or status transitions that are outside the behavioural spec.

7. EJS data contracts

Preserve the supplied layout, form field names, shared partials, and empty states. Supply every required local when rendering, including arrays when empty. Do not suppress a database failure by rendering an empty success-shaped page.

`res.render('services', locals)`

Local	Type / shape	Use
`pets`	Array of `{ id, name, species, ... }`	Owned-pet dropdown
`services`	Array of `{ id, name, price }`	Active service radio choices
`selectedPetId`	String ID or `''`	Match against `String(pet.id)`
`today`	`YYYY-MM-DD` string	Date input default and minimum
`messages`	Array of strings	Consumed success flash messages
`errors`	Array of strings	Consumed error flash messages
`session`	`req.session`	Navbar login/user display

The form submits `POST /services/book` with names `petId`, `serviceId`, and `bookingDate`. It does not submit an authoritative price.

`res.render('bookings', locals)`

Local	Type / shape	Use
`bookings`	Array of booking-history rows	History table or empty state
`messages`	Array of strings	Consumed success flash messages
`errors`	Array of strings	Consumed error flash messages
`session`	`req.session`	Navbar login/user display

Each booking-history row must expose:

Field	Representation
`booking_id`	Booking ID
`booking_date`	`YYYY-MM-DD` string, not an unformatted JavaScript `Date`
`pet_name`	Pet name
`service_name`	Service name
`booked_price`	Database decimal value
`booking_status`	Stored status string
`payment_status`	Stored status string, or `null` when no payment exists

The existing model also selects `payment_reference` and `created_at`; the provided table does not need to display them.

MySQL decimal values may arrive as strings. Preserve the stored decimal price for persistence; the supplied shells use `Number(value).toFixed(2)` only to display Singapore-dollar amounts. Use escaped EJS output for user/database text.

8. Navigation and integration boundaries

- Keep signed-in navigation links to `/services` and `/bookings`.
- Keep the service page's history link and the history page's booking link.
- Reuse the existing `/pets/new` link when a customer has no pets.
- Preserve login, pet, journal, admin, and shared-navbar behaviour.
- Do not replace the existing project structure or introduce a new framework.
- Do not rewrite the behavioural spec to accommodate implementation shortcuts.

9. Implementation and verification sequence

1. Read both specifications, the schema, and the provided shells.
2. Confirm the existing authentication, flash, database, and pet-query contracts.
3. Create or complete the model exports.
4. Create or complete the controller exports and render locals.
5. Wire the protected router into the existing application.
6. Connect the shells and navigation without redesigning them.
7. Run the application from the project root using `node app.js`. Restore dependencies with the existing package manifest only if they are missing.
8. Verify AC1-AC8 and the regression checks in the behavioural specification.

Integration check	Behavioural criterion / evidence
Valid booking, database price, pending linked records	AC1; browser result and database rows
Today/future dates; reject malformed, impossible, or missing inputs	AC2-AC3

Integration check	Behavioural criterion / evidence
Ownership and inactive/unknown service rejection	AC4-AC5; no new records
Failed write/commit handling and connection cleanup	AC6; isolated tests or suitable database mocks
Correct render fields and owner-filtered ordering	AC7; history for different customers
All three routes protected	AC8; signed-out redirects
Existing login and journals	Behavioural regression checklist

Use a controlled date fixture for repeatable tests. The Clara/Mira example date is not a permanent valid date. Do not change the computer clock or break a shared database to force test outcomes.

The current package does not provide an application test suite. Use available focused checks and manual browser/database evidence; report unexecuted checks as `Not run` / `Not assessed`, never as passing.

10. Completion evidence

An implementation is complete only when the backend files exist, the routes are registered, the provided pages render without missing locals, and the specified behaviour has been verified.

Report:

- Files created or updated and their wiring.
- Required database setup and application run instructions.
- Acceptance criteria checked, actual results, and supporting evidence.
- Any tests not run and remaining blockers.

These checks are requirements for the implementation exercise, not a claim that the application has already been tested. This document alone does not create or modify application code.